

Algorithmique et Programmation orientée-objet

Ch.8 – Tableaux dynamiques

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Listes

- **Introduction**

- Une **liste** est une séquence ordonnée d'éléments. Une liste peut contenir des éléments dupliqués.
- Une liste est un **type de données abstrait** (ADT – *Abstract Data Type*). Un ADT définit les services d'une structure de données sans imposer une implémentation particulière.
- Il existe plusieurs implémentations de l'ADT liste
 - `ArrayList` repose sur l'utilisation de *tableaux*.
 - `LinkedList` repose sur les *listes chaînées*.

Listes

- **Interface List**

- L'interface `List` du JCF définit les méthodes que toute implémentation d'une liste doit fournir. Un extrait de ces méthodes est fourni ci-dessous.

```
public interface List<E> extends Collection<E>
{
    boolean    add(E o);
    void       clear();
    boolean    contains(E o);
    E          get(int index);
    int        indexOf(E o);
    boolean    isEmpty();
    boolean    remove(int index);
    boolean    remove(E o);
    void       set(int index, E o);
    int        size();
}
```

Récupère un élément à partir de sa position absolue dans la liste (obligatoire).

Fournit le nombre d'éléments de la liste (obligatoire).

Notes :

- Toutes les méthodes ne doivent pas nécessairement être implémentées (voir documentation). Les méthodes optionnelles non implémentées doivent générer une `UnsupportedOperationException`.
- La méthode `add(E o)` retourne un booléen de façon à permettre de refuser l'ajout de certains éléments.

Tableaux Dynamiques

- Exemple

```
import java.util.ArrayList;
import java.util.List;

List<Carre> carres= new ArrayList<Carre>();
carres.add(new Carre(5, 7, -60, 5));
carres.add(new Carre(4, 1, 0, 2));
System.out.println("Taille = "+carres.size());

for (int i= 0; i < carres.size(); i++)
    carres.get(i).deplacer(Math.random(),
                           Math.random());
carres.set(0, new Carre(2, 2, 2, 2));
```

Le constructeur crée un tableau interne vide.

Note : il existe également un autre constructeur qui prend un paramètre **int** et crée un tableau de N éléments.

La méthode `size` retourne la taille actuelle du tableau.

La méthode `add` ajoute un élément à la fin du tableau. La taille du tableau interne est augmentée si nécessaire.

Le paramètre de type **E** est égal à **Carre**. Cela affecte le type de retour de `get`, ainsi que les arguments de `add` et `set`.

Tableaux Dynamiques

- **ArrayList vs tableau**

- **Tableau**

- Taille fixe
 - Cellules peuvent être de type primitif
 - Accès avec l'opérateur d'indexation []
 - Exception `ArrayIndexOutOfBoundsException`

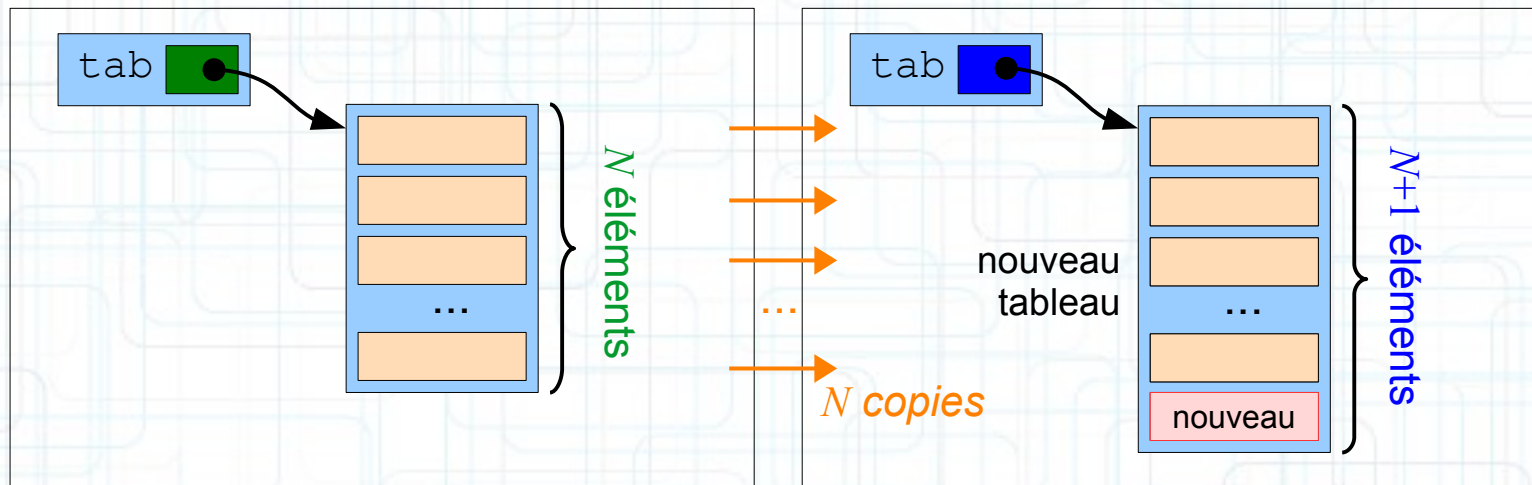
- **ArrayList**

- Taille variable
 - Éléments uniquement de type objet
 - Accès via `set()` et `get()`
 - Exception `IndexOutOfBoundsException`
 - Insertion/suppression d'un élément à une position arbitraire

Tableaux Dynamiques

- **Implémentation**

- Une tableau dynamique utilise un **tableau interne** (de taille fixe) pour stocker ses éléments.
- Lors de l'ajout d'un **nouvel élément**, il est possible qu'il n'y ait plus suffisamment de place dans le tableau interne. Auquel cas, il faut « **ré-allouer** » le **tableau** et y **copier** les éléments du tableau précédent.



- La ré-allocation et la copie des éléments prennent un temps proportionnel à la taille du tableau.

Tableaux Dynamiques

- Implémentation

```
public class MyArrayList {  
    protected Object[] tab = null; ..... référence vers  
                                         tableau interne  
    public void add(Object o) {  
        int oldSize = size();  
        Object[] newTab = new Object[oldSize + 1]; ..... ré-allocation  
        if (oldSize > 0)  
            System.arraycopy(tab, 0, newTab, 0, oldSize); ..... copie  
        newTab[oldSize] = o;  
        tab = newTab;  
    }  
    public Object get(int i) {  
        if (tab == null)  
            throw new IndexOutOfBoundsException();  
        return tab[i];  
    }  
    public int size() {  
        if (tab == null)  
            return 0;  
        return tab.length;  
    }  
}
```


Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Tableaux Dynamiques

- **Complexité de l'ajout**

- Intéressons-nous à la complexité de l'ajout d'un élément à la fin d'un tableau dynamique.
- Prenons les hypothèses suivantes
 - la **ré-allocation** du tableau se fait en un **temps constant**.
 - la **copie d'un nouvel élément** dans le tableau se fait en un **temps constant**.
- La **copie des éléments du tableau précédent** lors d'une ré-allocation nécessite la **copie des N éléments** du tableau précédent.
- Nous nous focalisons donc sur le nombre de copies en cas de ré-allocation.

Tableaux Dynamiques

- **Complexité de l'ajout**

- Supposons que l'on ajoute n éléments à une `ArrayList` initialement vide.
 - 1^{er} ajout : pas de copie
 - 2^{ème} ajout : 1 copie
 - 3^{ème} ajout : 2 copies
 - ...
 - $n^{\text{ème}}$ ajout : $n-1$ copies
- Notons $C(k)$ le nombre de copies effectuées par `add()` lors de l'ajout du $k^{\text{ème}}$ élément. Dans notre implémentation, nous avons

$$C(k) = k-1$$

Tableaux Dynamiques

- **Ajout des n premiers éléments**

- Quel est le **nombre total de copies** effectuées pour ajouter les n premiers éléments ?

$$C_{total}(n) = \sum_{k=1}^n C(k) = \sum_{k=1}^n k-1 = \sum_{k=0}^{n-1} k = \frac{n(n-1)}{2}$$

- Quel est le **nombre moyen de copies** effectuées par ajout pour les n premiers éléments ?

$$\bar{C}(n) = \frac{C_{total}(n)}{n} = (n-1)/2$$

Exemple :

k	$C(k)$	$C_{total}(k)$	$\bar{C}(k)$
1	0	0	0
2	1	1	1/2
3	2	3	1
4	3	6	3/2
5	4	10	10/5

Tableaux Dynamiques

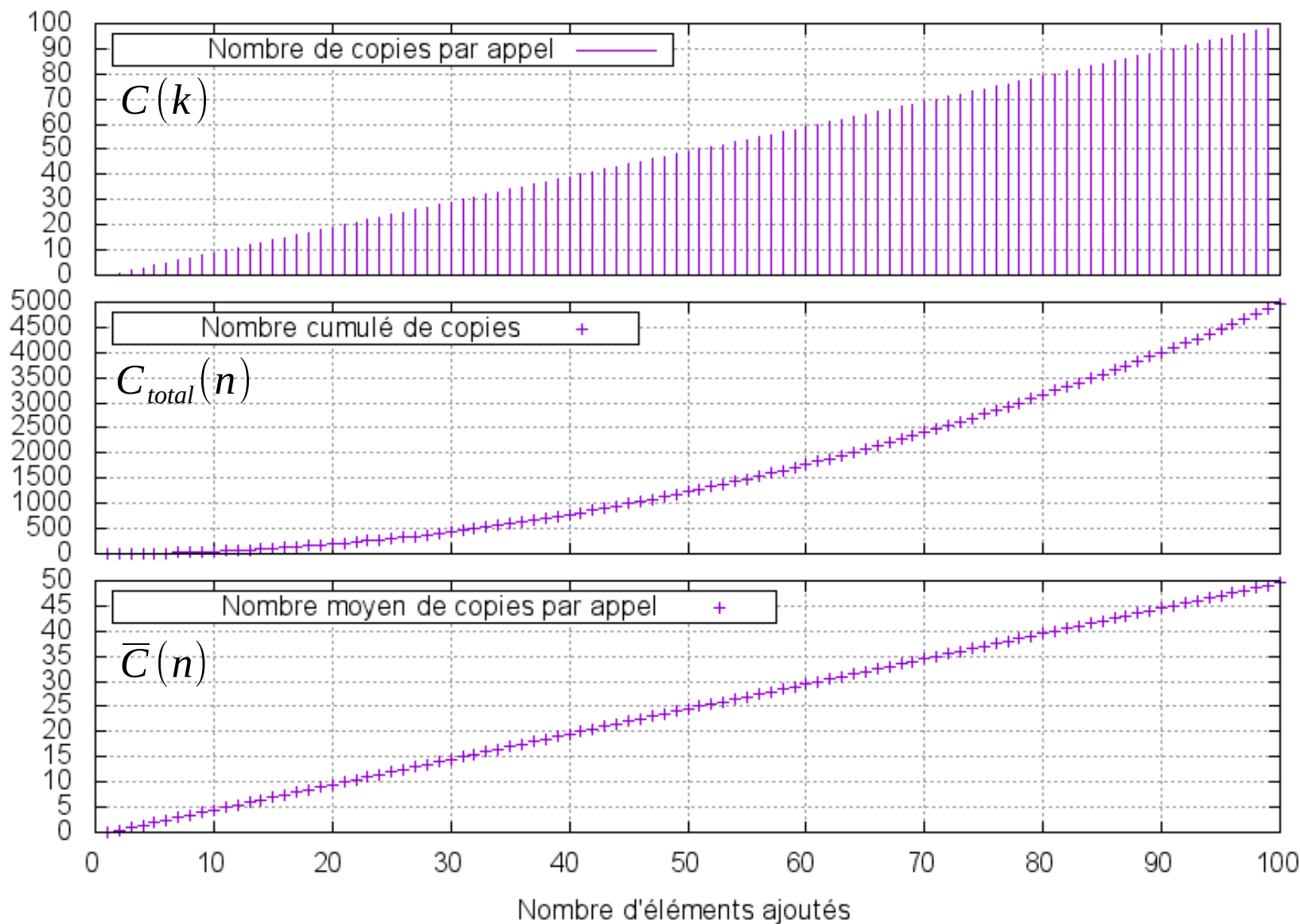


Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Tableaux Dynamiques

- **Complexité de add**

- La complexité temporelle de la méthode `add()` dans notre implémentation de `MyArrayList` **croît linéairement en fonction du nombre d'éléments** déjà dans le tableau !!!
- **Peut-on faire mieux ?**
 - En pratique, d'autres stratégies de croissance du tableau interne peuvent être utilisées.
 - **Stratégie additive** : ajouter à la taille du tableau une constante entière $a \geq 1$
 - **Stratégie multiplicative** : multiplier la taille du tableau par un facteur constant $b > 1$
- L'implémentation d'`ArrayList` de la librairie Java utilise une stratégie multiplicative.

Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Tableaux Dynamiques

- **Stratégie additive**

- Supposons que l'on ajoute n éléments à une `ArrayList` initialement vide avec une stratégie additive.
- Hypothèse : croissance de $a=10$ cellules
 - 1^{er} au 10^{ème} ajouts : pas de copie
 - 11^{ème} ajout : 10 copies
 - 12^{ème} au 20^{ème} ajouts : pas de copie
 - 21^{ème} ajout : 20 copies
 - 22^{ème} au 30^{ème} ajouts : pas de copie
 - ...

Tableaux Dynamiques

- **Stratégie additive**

- Avec cette stratégie, le nombre de copies pour le $k^{\text{ème}}$ ajout est égal à

$$C(k) = \begin{cases} k-1 & \text{si } \exists p \in \mathbb{N} : k = ap + 1 \\ 0 & \text{sinon} \end{cases}$$

Exemple :
($a=10$)

k	taille	$C(k)$	$C_{\text{total}}(k)$	$\bar{C}(k)$
1	$0 \rightarrow 10$	0	0	0
2	10	0	0	0
...				
11	$10 \rightarrow 20$	10	10	10/11
12	20	0	10	10/12
...				
21	$20 \rightarrow 30$	20	30	30/21
22	30	0	30	30/22
...				
31	$30 \rightarrow 40$	30	60	60/31
32	40	0	60	60/32

pires
cas

$k=a+1$

$k=2a+1$

$k=3a+1$

Tableaux Dynamiques

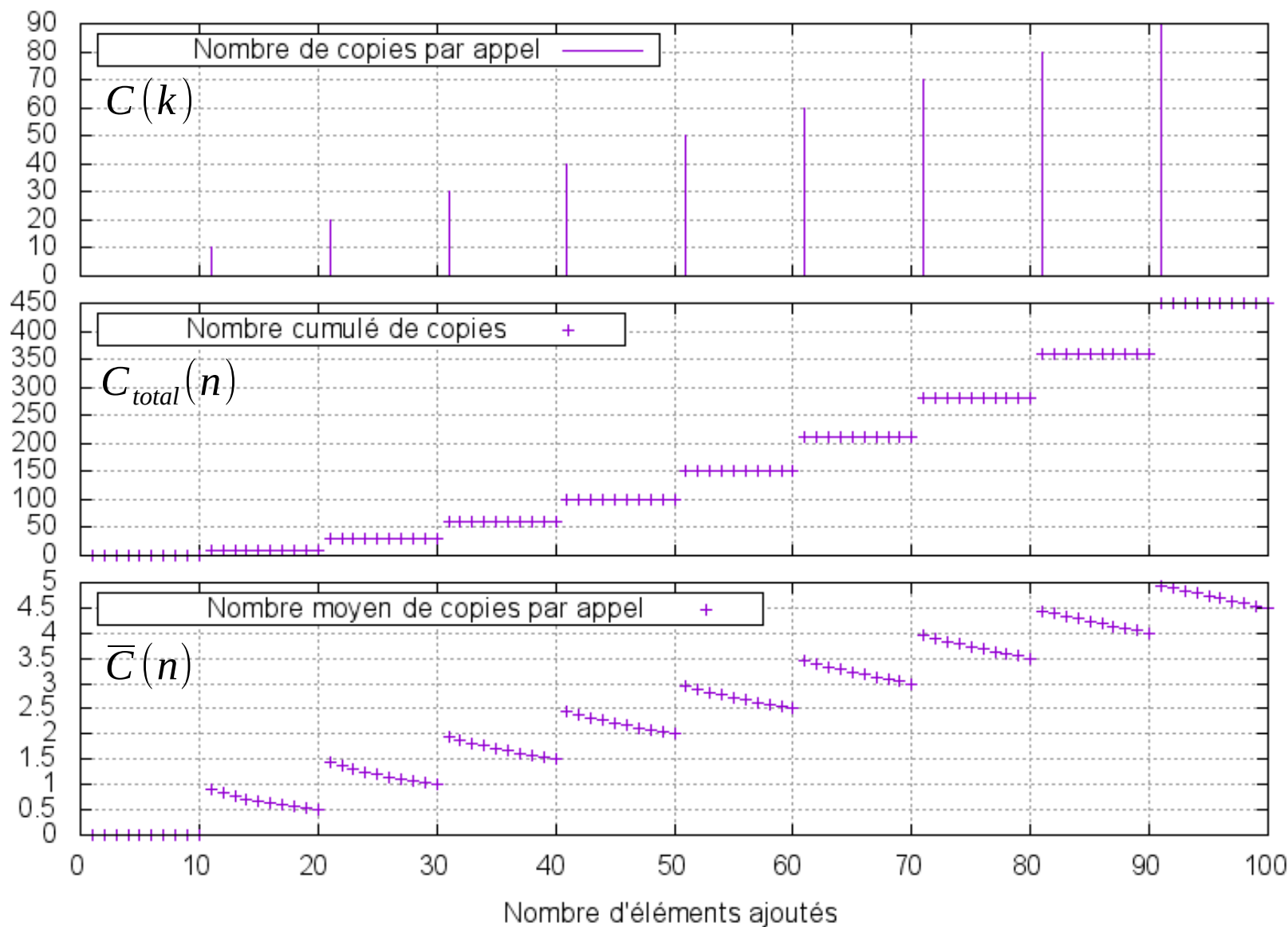
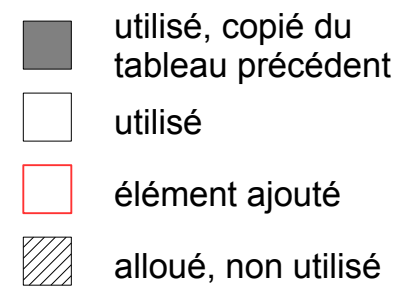


Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Tableaux Dynamiques

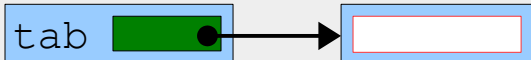
- Stratégie multiplicative**



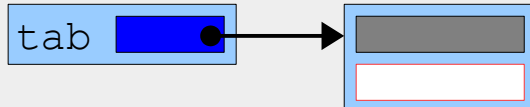
Situation initiale

tab null

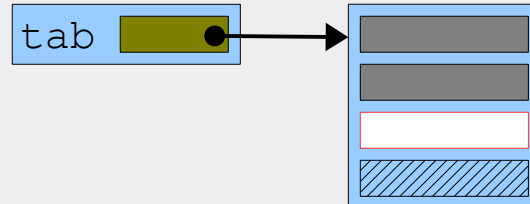
1^{er} ajout



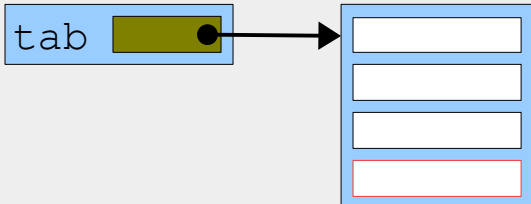
2^{ème} ajout → 1 copie



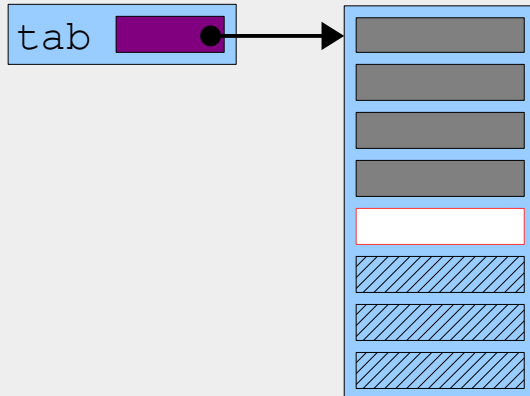
3^{ème} ajout → 2 copies



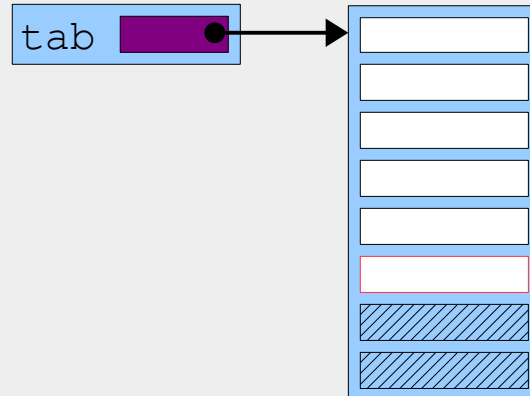
4^{ème} ajout



5^{ème} ajout → 4 copies



6^{ème} ajout



Tableaux Dynamiques

- **Stratégie multiplicative**

- Supposons que l'on ajoute n éléments à une `ArrayList` initialement vide avec une stratégie multiplicative.
- Hypothèse : facteur $b=2$
 - 1^{er} ajout : pas de copie
 - 2^{ème} ajout : 1 copie
 - 3^{ème} ajout : 2 copies
 - 4^{ème} ajout : pas de copie
 - 5^{ème} ajout : 4 copies
 - 6^{ème} au 8^{ème} ajouts : pas de copie
 - 9^{ème} ajout : 8 copies
 - 10^{ème} au 16^{ème} ajout : pas de copie
 - 17^{ème} ajout : 16 copies
 - ...

Tableaux Dynamiques

- **Stratégie multiplicative**

- Avec cette stratégie, le nombre de copies pour le $k^{\text{ème}}$ ajout est égal à

$$C(k) = \begin{cases} k-1 & \text{si } \exists p \in \mathbb{N} : k = b^p + 1 \\ 0 & \text{sinon} \end{cases}$$

Exemple :
($b=2$)

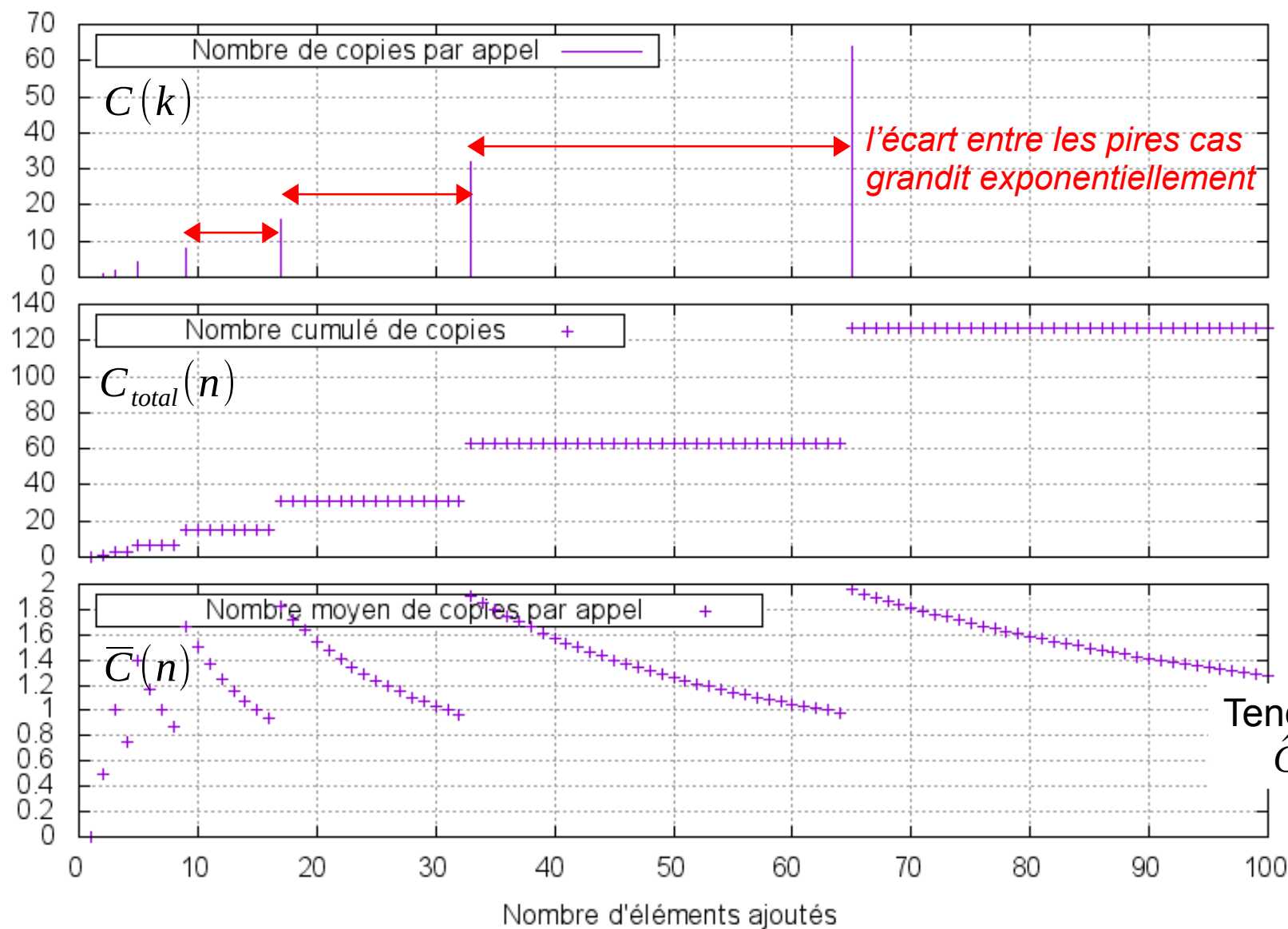
k	taille	$C(k)$	$C_{\text{total}}(k)$	$\bar{C}(k)$	
1	$0 \rightarrow 1$	0	0	0	
2	$1 \rightarrow 2$	1	1	$1/2$	$k=2^0+1$
3	$2 \rightarrow 4$	2	3	1	$k=2^1+1$
4	4	0	3	$3/4$	
5	$4 \rightarrow 8$	4	7	$7/5$	$k=2^2+1$
6	8	0	7	$7/6$	
7	8	0	7	$7/7$	
8	8	0	7	$7/8$	
9	$8 \rightarrow 16$	8	15	$15/9$	$k=2^3+1$
10	16	0	15	$15/10$	

pires
cas



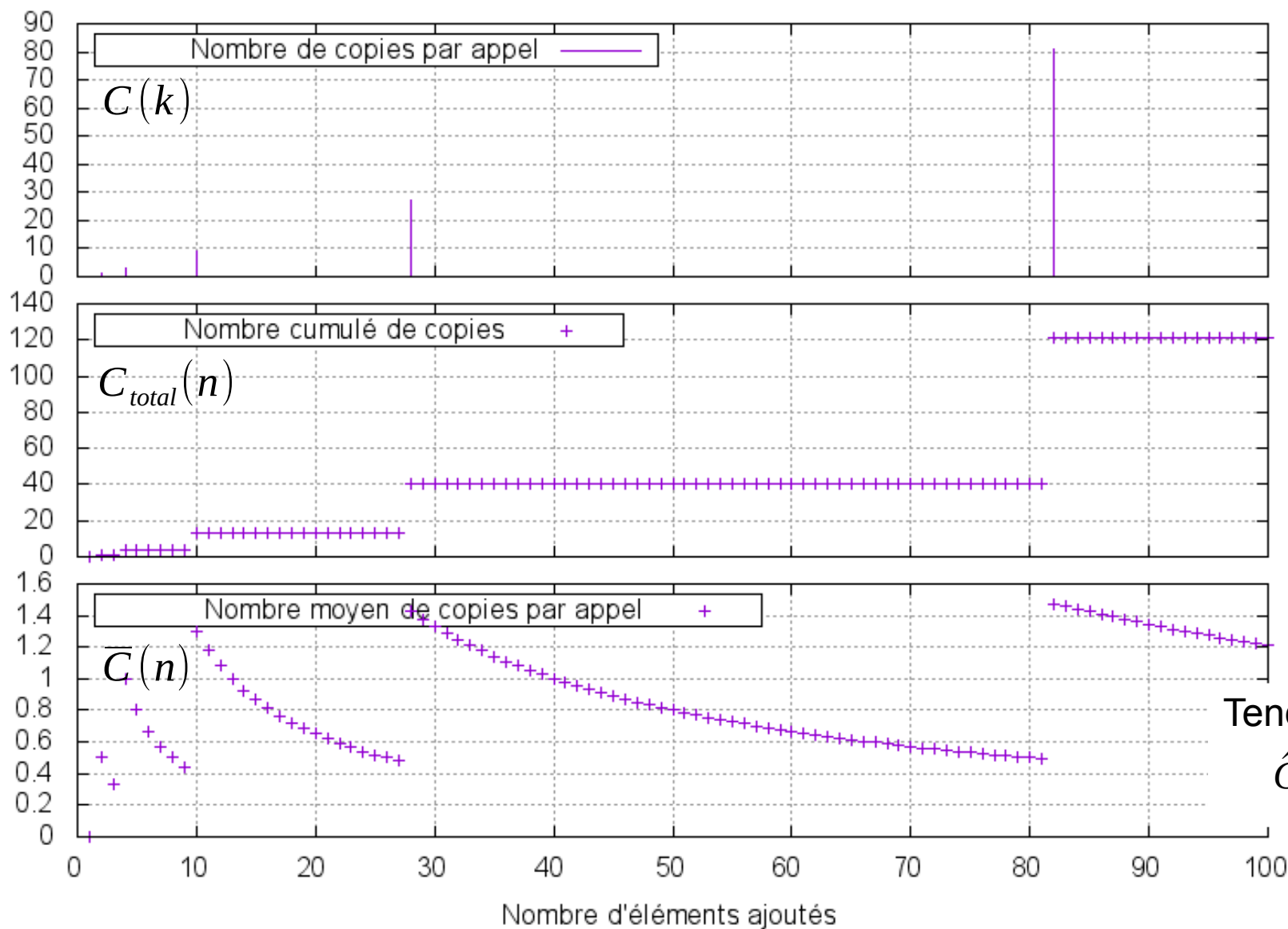
cas
 $b = 2$

Tableaux Dynamiques



cas
 $b = 3$

Tableaux Dynamiques



Tableaux Dynamiques

- **Stratégie multiplicative**

- Les **pires cas** arrivent lorsque $k = b^p + 1$, avec p naturel.
- Effectuons un changement de variable de k à p

$$C(k) = k - 1 \Rightarrow C(b^p + 1) = b^p$$

- Le **nombre total de copies** effectuées lors d'une séquence d'ajouts devient

$$C_{total}(b^p + 1) = \sum_{i=0}^p b^i = \frac{b^{p+1} - 1}{b - 1}$$

- Le **nombre moyen de copies** effectuées par ajout lors d'une séquence d'ajouts devient

$$\bar{C}(b^p + 1) = \frac{C_{total}(b^p + 1)}{b^p + 1} = \frac{1}{b - 1} \frac{b^{p+1} - 1}{b^p + 1}$$

Tableaux Dynamiques

- **Stratégie multiplicative**

- Le **nombre moyen de copies** a une limite finie lorsqu'une stratégie de croissance multiplicative est employée.

$$\hat{C} = \lim_{p \rightarrow +\infty} \bar{C}(b^p + 1) = \lim_{p \rightarrow +\infty} \frac{1}{b-1} \frac{b^{p+1} - 1}{b^p + 1} = \frac{b}{b-1}$$

- Par exemple
 - avec $b=2$, le nombre moyen de copies vaut 2
 - avec $b=3$, il vaut $3/2$
 - avec $b=3/2$, il vaut 3
- Note: avec une stratégie additive, cette limite est infinie.

Tableaux Dynamiques

- **Stratégie multiplicative**

- La complexité temporelle classique s'intéresse au pire des cas pour un **ajout individuel**.
 - La complexité est en $O(n)$ quelque soit la stratégie (additive ou multiplicative).
- Cependant, si on considère des **séquences de n ajouts**, certains ajouts sont coûteux et d'autres non. Le coût est amorti sur plusieurs ajouts.
 - La complexité pour une séquence de n ajouts est en $O(n)$ avec une **stratégie multiplicative**.
 - Elle est en $O(n^2)$ avec une **stratégie additive**.
- La complexité moyenne (amortie) d'un ajout individuel est en $O(1)$ avec une stratégie multiplicative.

Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Tableaux Dynamiques

- **Taux d'occupation**

- La stratégie multiplicative permet une complexité amortie constante. Cependant, **quel facteur multiplicatif b utiliser ?**
- Un second point important à considérer est le **taux d'occupation** du tableau interne.
 - La taille de celui-ci est toujours au moins égale au nombre d'éléments à stocker.
 - Lorsque la taille du tableau est supérieure au nombre d'éléments à stocker, un espace mémoire supplémentaire a été alloué mais n'est pas utilisé. Il s'agit d'une forme de gaspillage.
- Le **taux d'occupation** du tableau est mesuré comme le nombre d'éléments qu'il contient divisé par sa taille.

Tableaux Dynamiques

• Taux d'occupation

- Observons le **taux d'utilisation moyen** d'un tableau dynamique avec différentes stratégies de croissance

- Additive $a=1$: toujours 100 %
- Additive $a=10$: tend vers 100 % (avec assez d'éléments)
- Multiplicative $b=2$: **75 %** (pire = ~50%)
- Multiplicative $b=3$: **66 %** (pire = ~33%)
- Multiplicative $b=3/2$: **83 %** (pire = ~ 66%)

- Le choix du facteur b de la stratégie multiplicative est donc un **compromis** entre rapidité et espace mémoire consommé.

- $b=2 \rightarrow$ nombre moyen de copies = 2
taux moyen d'occupation = 75 %
- $b=3/2 \rightarrow$ nombre moyen de copies = 3
taux moyen d'occupation = 83 %



*le JCF utilise $b=3$
pour ArrayList*

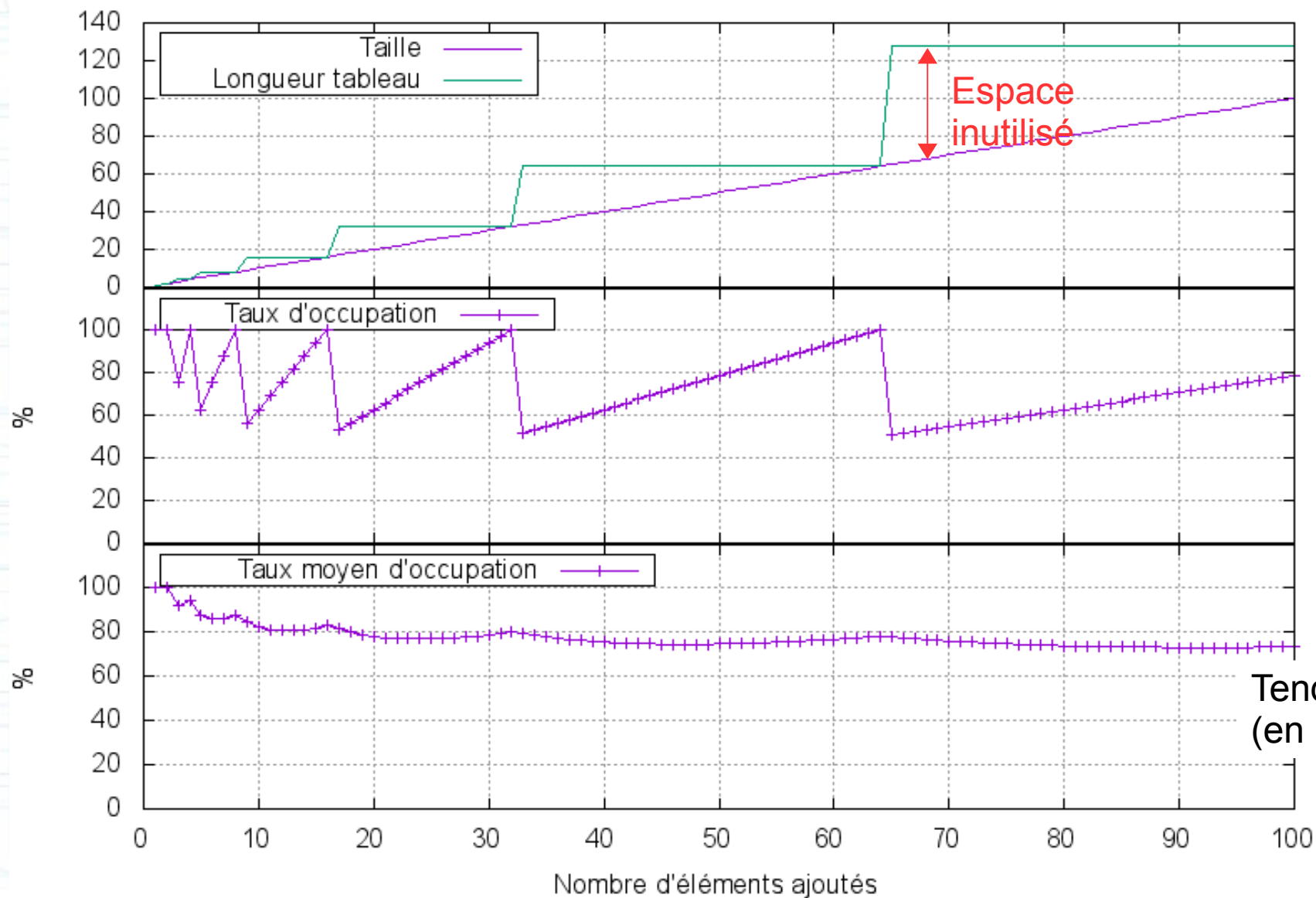
pire

$$\frac{ap+1}{a(p+1)}$$

$$\frac{b^p+1}{b^{p+1}}$$

cas
 $b = 2$

Tableaux Dynamiques



Tend vers 75 %
(en moyenne)

cas
 $b = 3/2$

Tableaux Dynamiques

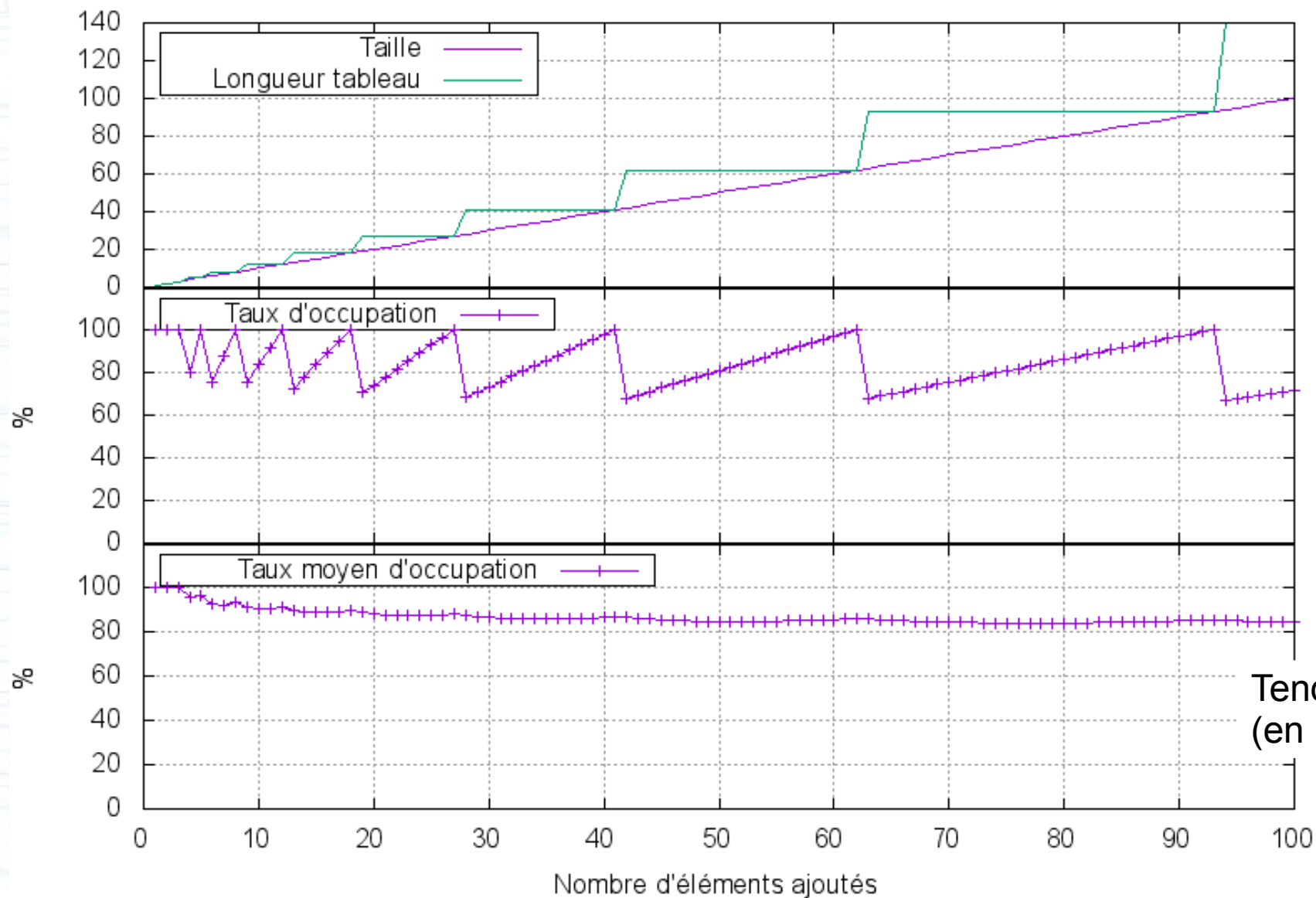


Table des Matières

- **Principes de base**
- **Complexité de l'ajout**
- **Stratégies de croissance**
 - Additive
 - Multiplicative
 - Taux d'occupation
- **Autres opérations**

Tableaux Dynamiques

- **Opérations d'insertion / suppression**
 - D'autres opérations fournies par la classe `ArrayList` sont moins efficaces: il s'agit des opérations **d'insertion et de suppression**. Le coût de ces opérations vient de la nécessité de déplacer un grand nombre d'éléments du tableau interne à `ArrayList`.

Tableaux Dynamiques

- **Insertion**

- La méthode `add(int index, E o)` de la classe `ArrayList` permet l'insertion d'une référence à une position spécifique de la liste.

- La méthode `add(E o)` que nous avons étudiée précédemment est un cas particulier d'insertion en fin de liste : `add(size(), o)`

- Exemple

```
List<Carre> carres = new ArrayList<Carre>();  
carres.add(new Carre(5, 7, -60, 5));  
carres.add(new Carre(4, 1, 0, 2));  
carres.add(1, new Carre(2, 2, 2, 2));  
for (int i= 0; i < carres.size(); i++)  
    System.out.println(carres.get(i));
```

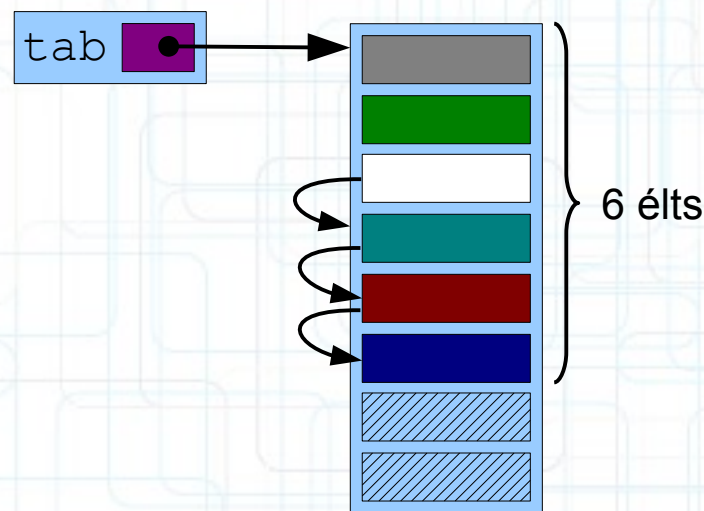
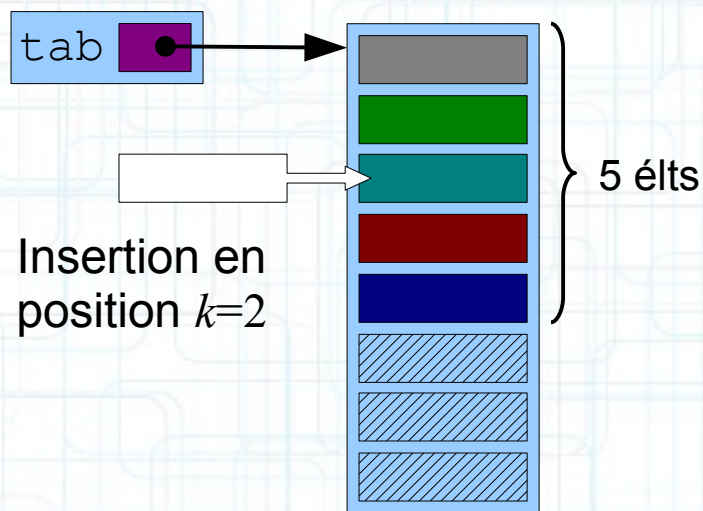
```
bash-3.2$ java ArrayListEx  
Carre(x=5.0;y=7.0)  
Carre(x=2.0;y=2.0)  
Carre(x=4.0;y=1.0)
```

Elément inséré à l'index 1

Tableaux Dynamiques

- **Complexité de l'insertion**

- L'insertion dans un tableau de taille N en position k nécessite le déplacement de $N-k$ éléments.

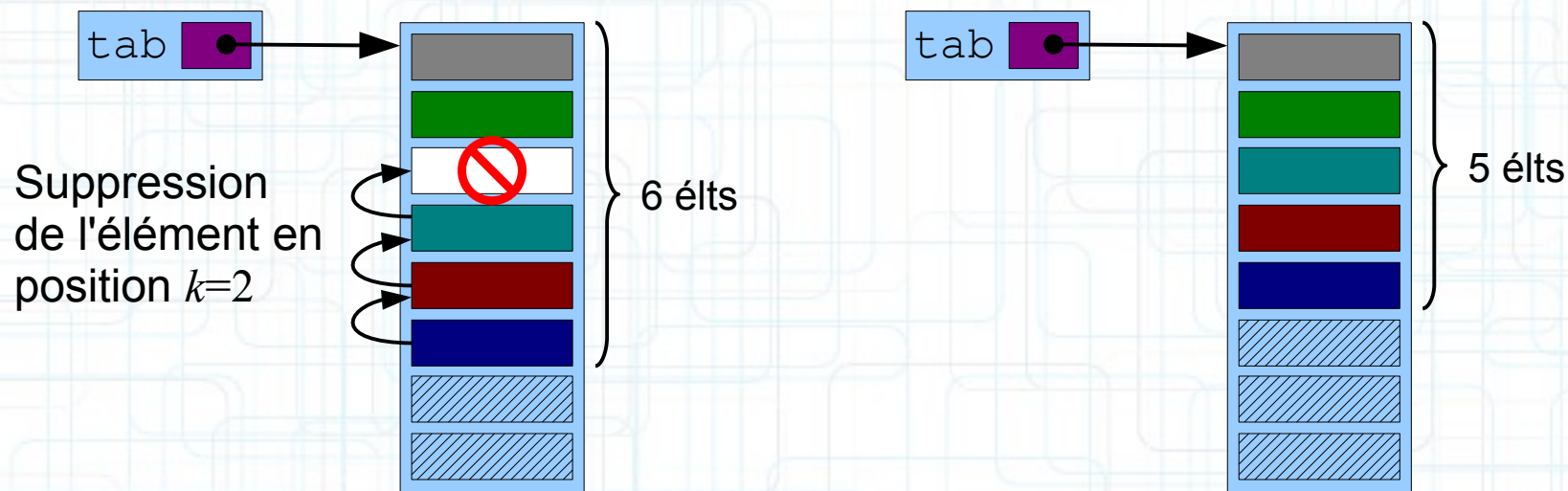


- La complexité temporelle de l'insertion (dans le pire des cas) est donc $O(N)$.
- Note : on ne considère pas le coût de l'expansion éventuelle du tableau. Est-ce un problème ?

Tableaux Dynamiques

- **Complexité de la suppression**

- La suppression d'un tableau de taille N en position k nécessite le déplacement de $N-k-1$ éléments.



- La complexité temporelle de la suppression (dans le pire des cas) est donc $O(N)$